

qc-docs

Standardize repository documentation so it matches the codebase that exists — nothing more.

Use when docs are missing, inconsistent, or trapped in chat; when a landing page and operational set are needed; for handoff and knowledge transfer.
Original: ~/.claude/skills/qc-docs/ · Do not use for packaging, feature work, or code hardening.
August 2026 · Port this document; do not weaken the laws.

Contents

- Purpose, two layers, and the document set
- Repo archetype matrix and layout
- Truth hierarchy
- First-pass vs delta, write order, checklist
- Public vs internal mode
- README standard and supporting-doc standards
- Style, change triggers
- Three quality gates and the validation matrix
- Severity, ledger, what not to do

1. Purpose

qc-docs standardizes repository documentation for production use, team handoff, and external readability **without changing application behavior**. Documentation must match the codebase that exists. Do not invent features, infrastructure, SLAs, or operating procedures that are not grounded in the repository or verified user input.

Triggers: production docs, standardize docs, documentation pass, repo docs, landing page, deployment docs, handoff docs, operator docs, runbook, knowledge transfer. **Do not use** for packaging/release polish, feature implementation, code hardening, CI hardening, runtime debugging, or speculative architecture without a real implementation.

Two layers

- **Layer 1 — GitHub landing page.** README.md is the entry point: problem, system, how to run, where deeper docs live. A new engineer should be oriented in under five minutes.
- **Layer 2 — Operational document set.** Dedicated files hold durable reference so the README stays readable. If a file is not applicable, do not create placeholder noise. The README must still point to the applicable set.

File	Purpose	Required when
README.md	Landing page and operator entry	Always
ARCHITECTURE.md	Structure, data flow, components, boundaries	Always
DEPLOYMENT.md	Env, deploy/run, config, rollback basics	Deployable systems or scheduled jobs
OPERATIONS.md	Day-2 runbook, health, monitoring, backup/restore	Services, pipelines, schedulers, persistent systems

File	Purpose	Required when
TROUBLESHOOTING.md	Known failure modes, symptoms, checks, recovery	Any non-trivial system
HANDOFF.md	Ownership, commands, critical files, known risks	Repos intended for another maintainer or team
SECURITY.md	Posture and disclosure/contact	Public or shared repos
CHANGELOG.md	Versioned change history	Releases or recurring milestones
API.md	Routes, contracts, payloads, auth	APIs, machine CLIs, SDKs
DATA_MODEL.md	Entities, schemas, lifecycle, file formats	Data- or integration-heavy systems

2. Archetype matrix and layout

Choose the closest archetype before writing. If a repo fits several, use the union of required docs. When in doubt, bias toward the more operational archetype. Public or release-managed repos should usually have SECURITY.md and CHANGELOG.md even if the table omits them.

Archetype	Signals	Required extra emphasis
Library / SDK	Reusable package, exports, no long-running runtime	Install, compatibility, public interface, releases. Docs: README, ARCHITECTURE, API, SECURITY, CHANGELOG.
CLI / local tool	argparse/click/typer, local I/O, batch outputs	Invocation, inputs, outputs, common paths. Plus TROUBLESHOOTING, HANDOFF.
API / service	HTTP, health, deployable runtime, persistent config	Purpose, deploy path, health, interface. Plus DEPLOYMENT, OPERATIONS, TROUBLESHOOTING, API, SECURITY, HANDOFF.
Scheduled job / pipeline	Cron, queue worker, generated artifacts	Schedule, inputs, outputs, reruns, recovery. Plus DEPLOYMENT, OPERATIONS, TROUBLESHOOTING, HANDOFF, DATA_MODEL.
Data / integration	Formats, schemas, connectors, ingest	Sources, schemas, lineage, artifact flow. Same operational set as pipelines.
Infra-only	Terraform, Bicep, Ansible, no app runtime	Environments, rollout, rollback, operational boundaries. Plus DEPLOYMENT, OPERATIONS, TROUBLESHOOTING, SECURITY, CHANGELOG, HANDOFF.

Canonical layout (normalize content first; move files only when benefit exceeds churn)

README.md, SECURITY.md, CHANGELOG.md at repo root. Everything else under docs/: ARCHITECTURE, DEPLOYMENT, OPERATIONS, TROUBLESHOOTING, HANDOFF, API, DATA_MODEL, plus docs/diagrams/, docs/examples/, docs/internal/ when needed. Keep the README short. Do not duplicate full runbooks in it. Root-level docs stay limited to the landing page and high-signal policies. Every document linked from the README must be canonical, not a draft or duplicate. If the repo already has a stable, well-linked structure, keep it.

3. Truth hierarchy

When sources disagree, use this order. Never guess.

1. Executable behavior: CLI help, route definitions, config loaders, tests that exercise the path, generated artifacts, observed runtime.
2. Source code and configuration: application code, package manifests, Dockerfiles, workflows, infra, env-var lookups.
3. Verified user-supplied operating facts (venue, ownership, unpublished constraints). If not inferable from code, label them as user-provided.
4. Existing documentation and comments.

5. Assumptions.

- If docs conflict with code, code wins.
- If code and tests conflict, investigate and document the ambiguity before writing definitive prose.
- User-supplied operating facts must not be presented as code-derived truth.
- If a fact cannot be verified, state it as unknown or provisional.

4. First-pass vs delta, write order

First-pass

Default when docs are missing, skeletal, or fail the structural check. Full sequence: audit, write bottom-up, README last, then validate.

Delta mode

Skip the full write when all three hold: the required doc set for the archetype exists; each doc has substantive repo-specific content (not a placeholder); the README links to the set and follows the required structure. Then: run the validation matrix, fix by severity, run quality gates, stop when gates pass. If validation reveals a missing required doc or a placeholder, fall back to first-pass for the affected docs only.

Plan top-down, write bottom-up

Plan from the landing page outward, but do not rewrite the landing page first. Order: ARCHITECTURE → DEPLOYMENT → OPERATIONS → TROUBLESHOOTING → HANDOFF → conditional docs (API, DATA_MODEL, SECURITY, CHANGELOG) → README → cross-reference validation. Deep docs establish operational truth; the README summarizes and routes; cross-links are only meaningful after the set exists.

Before writing: audit actual entry points, deployment model, operators, dependencies, commands that work, persistent artifacts. Preserve accurate existing material; do not rewrite for style alone. Build a truth map: landing vs deep reference vs runbook vs still-unknown.

Default execution checklist

Step	Action	Skip when
1	Audit codebase; classify archetype	Never
2	Create or normalize ARCHITECTURE.md	Never
3	DEPLOYMENT.md	Library/SDK; CLI with no deploy target
4	OPERATIONS.md	Library/SDK; CLI tool
5	TROUBLESHOOTING.md	Trivial projects only
6	HANDOFF.md	No team transfer planned
7	Conditional: API, DATA_MODEL, SECURITY, CHANGELOG	See archetype matrix
8	Restructure README as the landing page	Never
9	Validate cross-refs, commands, contradictions; quality gates	Never

5. Public vs internal mode

Choose visibility before writing. Mixing modes is how secrets and nicknames leak.

Mode	Use when	Allowed / restricted
Public	Repo is or may go public; external landing page	Public commands, sanitized architecture, supported workflows, high-level deploy shape. Restrict internal hostnames, dashboards, ticket queues, escalation contacts, environment nicknames, private topology.
Internal	Private repo; docs for operators	Environment names, state paths, dashboards, ownership, rollback, incident inputs, scheduler specifics. Still no secrets, tokens, or credentials.
Hybrid	Public landing + private runbooks	Public README and sanitized supporting docs; internal runbooks in docs/internal/ or another private location. Never mix internal-only detail into the public README.

On DEPLOYMENT, OPERATIONS, TROUBLESHOOTING, HANDOFF, and API, prefer a header block: Audience (engineers | operators | maintainers), Visibility (public | internal), Last verified against (commit, tag, or release). If the repo is public and deeper internal runbooks cannot live in it, say so and keep public docs sanitized.

6. README and supporting-doc standards

Required README structure

```
# Project Name
[1 short tagline sentence]
[Problem paragraph]
[Solution paragraph]
[Hero diagram only if it adds real value]
---
## Quick Start
## What It Does
## Architecture
## Repository Map
## Operations          (short; links to deeper docs)
## Documentation Map
## Validation          (tests/lint/health that actually exist)
> **Scope note.**
```

README rules

- H1 once, project name only. Every H2 preceded by ---. H3 inside H2 is allowed; no H4+.
- No badges, shields, emoji, or table of contents by default. No marketing, hype, or AI attribution.
- No invented roadmap language unless verified by repo docs or the user.
- Tables for structured material (commands, docs map, files, endpoints, env vars).
- Narrative stays short. Every command is real and repo-valid. Every file referenced exists or is created in the same pass.

Supporting documents — each has one job

Doc	Must answer	Recommended sections
ARCHITECTURE	Components, data movement, boundaries, sync vs scheduled vs external, outputs.	Overview, Execution Flow, Component Responsibilities, Data Flow, External Integrations, Key Constraints, Architecture Risks.
DEPLOYMENT	Environments, prerequisites, config/env, build/start/schedule/publish, rollback.	Model, Prerequisites, Configuration, Build and Release, Run/Deploy Steps, Rollback, Post-Deploy Verification.
OPERATIONS	Routine tasks, health, logs/state, cleanup/retention/schedule/rerun.	Overview, Routine Tasks, Health Checks, Monitoring Signals, State and Artifacts, Maintenance, Escalation Inputs.
TROUBLESHOOTING	Symptom-driven, not theory. Only issues supported by code, tests, or verified operator knowledge.	Table: Symptom Likely Cause Verify Fix.
HANDOFF	What the repo is for, who touches it, commands that matter, critical files, what is fragile, what to check first.	Purpose, Current Status, Ownership, First-Day Commands, Critical Files, Known Risks, Immediate Next Steps.
API	Per interface: route/command, method, inputs, outputs, errors, auth, example.	One contract per surface. Parameter types and return types, not just names.
DATA_MODEL	Real entities and real fields only. Prefer tables.	Entity, key fields, source of truth, lifecycle, file/API origin.

7. Style and change triggers

Voice

- Direct, technical, declarative. Short paragraphs. No tutorial filler, no sales tone, no AI provenance.
- Hedge only when uncertainty is real and stated. Prefer explicit limitations over vague reassurance.
- Inline code for commands, paths, env vars, field names. Tables for structured reference.
- Constants in tables: show the value as it appears in code; describe the effect in Purpose, not in Value (PENALTY = 0.15, not -0.15).
- Claims about where code lives (“all X are in Y”) must name specific items or be grep-verified. Blanket claims rot.

Documentation updates ship with the code changes that require them. If a trigger fires, update the corresponding docs in the same pass or explain why no change was needed.

Change in the repo	Docs to review
CLI command, subcommand, or flag	README, API/CLI section, OPERATIONS, HANDOFF
Env var, config file, or default	DEPLOYMENT, OPERATIONS, README Quick Start, TROUBLESHOOTING if failures change
Endpoint, payload, response, or error contract	API, DATA_MODEL, README, TROUBLESHOOTING
Output dir, artifact, state file, retention	OPERATIONS, TROUBLESHOOTING, HANDOFF, ARCHITECTURE if flow changes
Deploy model, container, infra, hosting	DEPLOYMENT, ARCHITECTURE, OPERATIONS, SECURITY
Scheduler cadence or recurring run	OPERATIONS, DEPLOYMENT, TROUBLESHOOTING, HANDOFF
Ownership, support, escalation	HANDOFF, OPERATIONS
Core entity, schema, state model, file format	DATA_MODEL, ARCHITECTURE, README
Public release or visibility change	README, SECURITY, CHANGELOG, plus visibility review across all docs

8. Quality gates and validation matrix

A production-docs pass is complete only when all three gates are green. Polished prose with a red gate is not done.

Gate	Pass	Fail
1 Completeness (coverage, not style)	Archetype identified; required set exists; every critical concern has a canonical home (entry, commands, config, env, interfaces, outputs, state, deploy, ops, failures, ownership); README points to deeper docs; no placeholders.	Required doc missing; concern exists only in comments/chat; README implies a complete story while deeper docs do not.
2 Accuracy (verification, not plausibility)	Commands, env, interfaces, outputs, deploy claims checked against code or verified user facts. Unknowns labeled. Last verified against headers on rewritten operational docs.	Docs contradict code/tests/startup; documented command/flag/endpoint/env/path does not exist; generic cloud guidance; user facts presented as code-derived.
3 Clarity (reader experience)	README answers what / how to run / where next in five minutes. Each doc has one job. Structured material in tables or steps. Troubleshooting is symptom-driven. Audience/visibility clear.	README is the full runbook; docs repeat or contradict; readers cannot tell which doc is canonical; instructions buried in narrative.

Convergence: multiple passes are expected if the first pass introduces errors (wrong import in a new API.md). Each pass should find fewer and lower-severity issues. **Stop when** all three gates pass and the current pass found only P3 issues or none. Do not run extra passes for style. **Warning:** new P1/P2 that the previous pass should have caught means the verification process has a gap (often non-markdown assets or blanket claims). Investigate the blind spot before continuing.

Validation matrix (minimum checks)

Check	Compare against	Minimum standard
Commands	README, DEPLOYMENT, OPERATIONS, HANDOFF vs CLI/scripts	Every documented command is real; run it or mark unverified.
Environment variables	DEPLOYMENT, OPERATIONS, API vs code lookups	Each var has a real source; required vs optional is clear.
Interfaces	API, README vs routes/CLI/SDK	Methods, paths, parameter types, return types, flags, errors match. dict vs list[X] is a blocker; list of str vs list[Dataclass] is a blocker.
Outputs / artifacts	ARCHITECTURE, OPERATIONS, TROUBLESHOOTING vs generated paths	Paths, names, meanings accurate.
Deployment path	DEPLOYMENT, README, OPERATIONS vs Docker/workflows/infra/startup	One consistent story, no contradictory paths.
Links	README and docs cross-links vs filesystem	No broken links; every referenced file exists.
Cross-doc consistency	All docs against each other	Entry points, env, deploy model, ownership do not contradict.
Visibility	Mode vs content	Public docs have no internal-only detail; internal docs still have no secrets.
Unknowns	All docs	Unknown facts labeled, not guessed.
Embedded assets	SVG/PNG/diagram labels and alt text vs prose	Rendered numbers and names match prose; alt text matches render.
Requirements mapping	SCOPE.md or equivalent if present	Every item in exactly one category: covered, partial, not yet, out of scope.
Code examples	Every code block vs source	Imports resolve; parameter names/types correct; attributes exist. Mentally execute: no AttributeError / ImportError / TypeError.
Dataclass / model fields	API/schema tables vs class definitions	Every documented field exists; no ghost fields; types match. Check secondary/nested dataclasses — that is where ghosts hide.
Status tables	Risk/known-issue lists vs current code	Fixed issues marked resolved. An active-risk entry for a resolved bug causes fake workarounds.

Code examples, dataclass fields, and interfaces need line-by-line reading against source. For repos with API.md or integration guides, dispatch **parallel subagents — one per doc file** — to catch ghost fields, type mismatches, stale examples, and resolved issues still listed as active. Surface-level checks consistently miss these because they require the doc and the class at the same time.

9. Severity, ledger, what not to do

Sev	Category	Examples	Priority
P1	Feature claims that contradict code	Doc says “not implemented” but it ships; documents a bug that is fixed; omits a major export	Fix immediately — misleads users and integrators
P2	Wrong references to code artifacts	Wrong module names, constant values, signatures, dataclass locations, broken links, missing imports in examples	Fix in the same pass
P3	Stale counts and cosmetic prose	Test counts, module counts, minor wording, H2 separators	After P1/P2; skip on delta unless trivial

Write `.qc-findings/qc-docs.json` to the shared schema (skill: qc-docs). Finding IDs like D2.1. Verdict mapping: zero findings → READY; only P2/P3, all fixed or deferred_because set → READY_WITH_DEBT; any P0/P1 with fixed:false and no

deferred_because → NOT_READY.

What not to do

- Do not create empty template files with no repo-specific content.
- Do not copy generic cloud guidance unrelated to the actual deployment model.
- Do not document endpoints, commands, or env vars that are not present.
- Do not leave contradictory instructions across files.
- Do not let the README become longer than the combined value of the deeper docs warrants.
- Do not rewrite code comments as a substitute for real documentation.

Relationship to the rest of the QC suite

- qc-packaging / production-packaging: packaging, release polish, repository hygiene beyond documentation.
- qc-hardening / production-hardening: defect reduction and production-risk reduction in code.
- qc-docs / production-docs: documentation structure, clarity, handoff quality, operational standardization.
- Do not run this skill as a substitute for either of the others.